

요청하신 넷플릭스 클론 코딩(넷뿌러졌스) 프로젝트의 분석 보고서입니다. 제공해주신 소스 코드와 README 문서, 그리고 Next.js App Router의 특성을 종합하여 전체적인 페이지 구성, 컴포넌트 설계 방식, 인증 흐름 등을 상세히 분석했습니다.



넷뿌러졌스 프로젝트 구조 및 아키텍처 분석 보고서

1. 프로젝트 개요 및 아키텍처 특징

본 프로젝트는 **Next.js 14 (App Router)** 기반으로 구축된 넷플릭스 클론 웹 애플리케이션입니다. **React Query**, **Tailwind CSS** 등의 모던 스택을 활용하며, 외부 API인 ****TMDB(The Movie Database)****를 통해 영화 및 TV 프로그램 데이터를 연동하고 있습니다.

특히 인증(Authentication)과 데이터베이스 처리에 있어 ****Firebase(인증 및 Realtime DB)****와 ****NextAuth(Auth.js)****를 결합한 하이브리드 구조를 채택하고 있는 것이 가장 큰 특징입니다. 이를 통해 클라이언트 사이드의 소셜 로그인(구글, 깃허브) 제어와 서버 사이드의 세션 관리를 효과적으로 분리하고 있습니다.

2. 페이지 단위 라우팅 구조 분석 (Page-level Routing)

Next.js 14의 라우트 그룹(Route Groups) 기능인 (폴더명) 컨벤션을 적극 활용하여, URL 경로에 영향을 주지 않으면서 로그인 전/후의 페이지와 레이아웃을 논리적으로 분리했습니다.

2.1 인증 전 라우트 그룹: (beforeLogin)

사용자가 인증을 거치기 전 접근하는 페이지들로 구성되어 있습니다.

- /login** (로그인 페이지): 이메일/비밀번호 기반 로그인 창 및 소셜 로그인 버튼 제공.
- /signup** (회원가입 페이지): 회원가입 폼 및 멤버십 안내.
- 특징: 이 그룹 내의 작업들은 주로 사용자 입력을 받고 **Firebase Auth**를 통해 인증을 시도하는 클라이언트/서버 액션으로 구성됩니다. **signup/_lib/signup.ts**를 보면 Next.js의 **Server Actions("use server")**를 이용해 폼 데이터를 서버에서 안전하게 처리하고, **Firebase** 유저 생성 후 **TMDB** 세션을 발급받아 **Realtime DB**에 저장하는 복합적인 로직을 수행합니다.

2.2 인증 후 라우트 그룹: (afterLogin)

로그인한 사용자만 접근 가능한 메인 서비스 영역입니다.

- **/browse** (홈/모든 콘텐츠): 인기 영화, TV 시리즈, 최신 콘텐츠 등 주요 미디어 리스트 출력.
- **/my-list** (내가 찜한 콘텐츠): TMDB 계정과 연동하여 사용자가 찜한 콘텐츠 목록 노출.
- **/profile** (프로필 및 설정): 계정 정보 관리 및 프로필 설정 페이지.
- 보안 제어 (**Middleware**): [src/middleware.ts](#) 파일에서 NextAuth의 세션을 검증하여 세션이 없는 사용자가 **/my-list**, **/profile** 관련 경로로 진입하려 할 때 강제로 **/login** 페이지로 리다이렉트시키는 보안 처리가 구현되어 있습니다.

3. 컴포넌트 사용 방식 및 데이터 패칭 전략

Next.js App Router의 핵심인 ****서버 컴포넌트(RSC)****와 ****클라이언트 컴포넌트(RCC)****를 목적에 맞게 분리하여 사용 중입니다.

3.1 클라이언트 컴포넌트 ("use client") - 상호작용 중심

- 대표 컴포넌트: `GithubButton`, `GoogleButton` (`/login/_component/`)
- 사용 방식: 브라우저 API나 이벤트 리스너(`onClick`)가 필요한 UI 요소에만 "use client" 지시어를 선언했습니다.
- 구현 로직: 버튼 클릭 시 `Firebase SDK`의 `signInWithPopup`을 호출하여 팝업창을 띄우고, 로그인 성공 시 `Firebase Realtime DB(users/{uid})`에 유저의 `TMDB` 세션이 존재하는지 확인(없으면 발급 후 저장)한 뒤, `next-auth/react`의 `signIn` 함수를 호출해 `Next.js` 서버 측 세션을 생성하는 정교한 비동기 로직을 컴포넌트 내부에 품고 있습니다.

3.2 서버 컴포넌트 및 서비스 레이어 - 데이터 중심

- 대표 모듈: [src/services/contents.ts](#), [src/app/\(afterLogin\)/_lib/tmdb-api.ts](#)
- 사용 방식: 콘텐츠 상세 정보(`getDetail`), 즐겨찾기 목록(`getFavorite`), 카테고리별 영화 목록(`getPopularMovies` 등)을 가져오는 로직은 브라우저를 거치지 않고 서버에서 직접 `TMDB API`와 통신합니다.
- 최적화: `fetch API` 사용 시 `{ cache: "no-store" }` 옵션을 부여하여 실시간성이 중요한 데이터(현재 상영작 등)가 캐싱되지 않고 항상 최신 상태를 유지하도록 설계되었습니다. 서버에서 데이터를 모두 패칭한 후 완성된 `HTML`을 클라이언트에 내려주므로 초기 로딩 속도(SEO)와 성능 면에서 이점을 가집니다.

4. 인증 및 데이터베이스 연동 흐름 요약

이 프로젝트의 가장 돋보이는 설계는 인증 흐름입니다. 보통 하나만 사용하는 `Firebase`와 `NextAuth`를 함께 활용했습니다.

1. 클라이언트 로그인 시도: 유저가 구글/깃허브 버튼 클릭 시 `Firebase` 팝업으로 로그인.
2. **DB 검증 (Firebase RTDB)**: 로그인된 `Firebase UID`를 기반으로 `DB`를 조회하여, 기존에 발급받은 `TMDB` 세션 아이디가 있는지 확인.

3. 세션 융합 (**NextAuth**): [auth.ts](#)에 정의된 **Credentials Provider**를 활용. **Firebase**에서 확인된 유저 이메일과 **TMDB** 세션 값을 **NextAuth** 쪽으로 넘겨주어(`signIn("credentials", ...)`), 최종적으로 **Next.js** 애플리케이션 내에서 접근 가능한 쿠키 기반 세션(`session.user.name`에 **TMDB** 세션 아이디를 저장하는 구조)을 발급합니다.

5. 전체 폴더 구조 관계도 (Tree Structure)

분석된 코드 및 리포지토리 구성 방식을 토대로 재구성한 프로젝트 폴더 논리 구조도입니다. 라우팅과 로직의 관계를 한눈에 파악하실 수 있습니다.

netflix-clone-main/

```
|
|
├── README.md          # 프로젝트 소개 및 배포 링크
├── pull_request_template.md # PR 컨벤션 템플릿
|
├── src/               # 애플리케이션 메인 소스코드
|   |
|   ├── firebase.ts    # Firebase 초기화 및 환경변수 설정
|   ├── auth.ts        # NextAuth(Auth.js) 인증 로직 (Google, Credentials)
|   ├── middleware.ts  # 라우터 접근 제어 (비로그인자 my-list, profile 접근 차단)
|   |
|   ├── app/           # Next.js 14 App Router 진입점
|   |   |
|   |   ├── (beforeLogin)/    # [라우트 그룹] 인증 전 접근 페이지
|   |   |   |
|   |   |   ├── login/        # 로그인 라우트 (/login)
|   |   |   |   |
|   |   |   |   ├── _component/  # 로그인 페이지 전용 컴포넌트
|   |   |   |   |   ├── github-button.tsx # 클라이언트 로그인 로직
|   |   |   |   |   ├── google-button.tsx
|   |   |   |   |   └── page.tsx
|   |   |   └── signup/        # 회원가입 라우트 (/signup)
|   |   |       |
|   |   |       ├── _lib/
|   |   |       |   ├── get-tmdb.ts # TMDB 세션 토큰 발급/검증 유틸
|   |   |       |   └── signup.ts  # [Server Action] 회원가입 서버 로직
```

```
| | └─ page.tsx
| |
| └─ (afterLogin)/      # [라우트 그룹] 인증 후 접근 페이지
| | └─ _lib/           # 인증 후 라우트 공통 모듈
| | | └─ tmdb-api.ts   # TMDB 상세 정보 및 즐겨찾기 API Fetching
| | | └─ tmdb-config.ts # TMDB Authorization 헤더 설정 (옵션)
| | └─ browse/         # 영화 목록 홈 (/browse)
| | └─ my-list/        # 내가 찜한 콘텐츠 (/my-list)
| | └─ profile/        # 프로필 설정 (/profile/*)
| |
| └─ layout.tsx        # 최상위 글로벌 레이아웃
|
└─ components/        # 전역적으로 재사용되는 공통 UI 컴포넌트
└─ config/            # 글로벌 설정 파일 (상수 등)
└─ hooks/             # 커스텀 훅 (React Query 훅 등)
└─ model/             # TypeScript 타입 및 인터페이스
| └─ media.ts         # 영화(Movie), 시리즈(Tv), 비디오(Video) 타입 정의
└─ services/          # 외부 API 호출 전용 서비스 레이어
    └─ contents.ts    # TMDB 인기작, 개봉작, 트렌드, 검색 Fetch 함수 모음
```